# SGLang self-benchmark journey

One campaign cell with two benchmark points, followed from AISim's plan to a published profile row. The original design runs in lane A and the in-scheduler variant in lane B, so each stage shows what the two share and where they part. The sections after the walkthrough compare version exposure, list the gaps lane B closes, settle the seven that were still open, and set out the AISim launch layer.

SHARED

What is measured, the point geometry, and every validity rule.

DIFFERENT

Where the controller runs (stages 2 and 3), which interval is timed (7 and 9), and which artifact schema is written (10 and 12).

STANDING

Lane B is a design read from source at two SGLang tags. Every open design question is now decided. It has not been prototyped: seven acceptance checks on a GPU remain.

**A · Original design**

A standalone runner outside the scheduler, as written in the design doc of 10 November 2026.

**B · In-scheduler variant**

A driver in the Dynamo worker sends requests through the engine's front door and stages them with SGLang's input blocker. A recorder inside the real SGLang scheduler logs and times the batches.

**Revised.** A second source reading changed two decisions. Hold and release now use SGLang's input blocker, because the earlier route, collective RPC, reaches only one DP rank under DP attention. The AISim work is now a launch interface with one implementation per backend, not a backend table. Frames 2, 7 and 8, the DP branch and the closing sections carry the changes.

## The specimen

The values are the design doc's own worked example (page 3), so each frame maps back to it. They stay fixed through the whole walkthrough.

| | |
|---|---|
| Cell and attempt | `C1`, attempt `a1`, SGLang v0.5.23, TP 1, no DP, page size 16 |
| Point `P`, cached prefill | 2 requests, 256 new tokens (128 each), 1,024 past KV tokens (512 each) |
| Point `D`, steady decode | 2 requests, 0 new tokens, 1,024 past KV tokens (512 each) |
| Requests for `P` | Seeds `S1`, `S2` (512 tokens). Measured `M1`, `M2` (640 = 512 prefix + 128 suffix). |
| Requests for `D` | `D1`, `D2` with 511-token prompts |
| State fields | `cache`: tokens held in the prefix cache per seed. `live`: requests holding KV, with their sequence length. `kept`: retained samples. `done`: points completed. |

## Route map

| # | STAGE | A · ORIGINAL | B · IN-SCHEDULER | STATUS |
|---|---|---|---|---|
| 1 | Plan and freeze | AISim plan, attempts, resume | Same, with the vLLM-only gates listed | Same |
| 2 | Launch | New launcher for a standalone process | The serving worker in benchmark mode | Changed |
| 3 | Initialize | Adapter replays the `load_model` order | Stock scheduler construction | Changed |
| 4 | Limits and grid | Pool and graph limits | Adds scheduler limits, reuses Dynamo's point schema | Same rule |
| 5 | Global warmup | Warm shots through the adapter | One real request through the scheduler | Same rule |
| 6 | `P` seed | Direct forward plus native cache builder | Seed requests through the stock path | Same state |
| 7 | `P` measure | Operation wall time or DeviceTimer | Scheduler wall time, launch to output | Changed |
| 8 | `D` seed and admission | Direct extend, first decode | Stock prefill, first decode step | Same arithmetic |
| 9 | `D` steady step | Second direct decode | Next scheduler loop iteration | Changed |
| 10 | Write evidence | New records and recorder | Dynamo rank-artifact schema v2 | Changed |
| 11 | Finalize | Exit, disposable worker | Flush, write, stop. Serving handoff possible. | Same in practice |
| 12 | Validate and publish | Backend extension, new timing policy | Existing validator and reader policy, behind a per-backend launch layer | Changed |

## Walkthrough

In each frame, the grey band holds what both designs do. The two lanes below it hold only what differs.

FRAME 1 OF 12

### AISim plans and freezes the cell

Same

Before `plan=none`

After `plan_sha256=H attempt=a1 points=[P, D]`

BOTH DESIGNS

Planning, attempts, smoke, resume and retries are AISim's existing machinery. The plan hash and attempt identity stay with AISim.

| | A · Original | B · In-scheduler |
|---|---|---|
| New work | "SGLang admission, launch arguments, probe and producer contract" (design doc, page 4). | The same work, named gate by gate: backend choices (`cli.py:27`), planner gate (`planner.py:731`), launch builder (`fpm_builder.py:72`), runtime probe (`runtime_probe.py:115`). |

FRAME 2 OF 12

### The worker is launched

Changed

BOTH DESIGNS

One worker per cell. AISim stops it after collection.

| | A · Original | B · In-scheduler |
|---|---|---|
| Command | A new benchmark process with its own launch arguments and producer contract. | The serving worker, `python -m dynamo.sglang` with `--benchmark-mode agg`, using the flag names the vLLM benchmark already has (`--benchmark-points-file`, `--benchmark-warmup-iterations`, `--benchmark-timeout`, `--benchmark-output-path`). |

| | A · Original | B · In-scheduler |
|---|---|---|
| How the controller gets in | It is the process. | In two parts. A driver runs in the Dynamo worker after the engine is built and before it registers for serving. A recorder, a `Scheduler` subclass, is installed by SGLang's plugin loader, which `run_scheduler_process` calls first. Dynamo's existing scheduler wrapper needs no change. Before it builds the engine, benchmark mode also sets `SGLANG_ENABLE_COLOCATED_BATCH_GEN`, which turns on SGLang's input blocker. |
| Request path | The runner builds requests and batches itself. | `Engine.async_generate` with `input_ids`, `rid` and `routed_dp_rank`: the public front door, the same in both tags. |

Evidence: load_plugins, called at the top of run_scheduler_process; class replacement; async_generate; `dynamo/sglang/nixl_telemetry.py:196`; the input blocker switch.

FRAME 3 OF 12

## The runtime initializes

Changed

After `cache={} live=[] kept=[] done=0/2`

BOTH DESIGNS

The same model, memory pools, attention backends and CUDA graphs get built.

| | A · Original | B · In-scheduler |
|---|---|---|
| Init order | The adapter replays `one_batch.load_model`: parallel runtime, layer runtime, `ModelRunner`, pools, attention backends, graphs. The adapter owns this order and has to track it across SGLang releases. | Stock `Scheduler.__init__` inside `run_scheduler_process`. SGLang owns the order. |
| Prefix cache | Built separately with the native cache builder, because `one_batch.extend` uses a dummy cache. | The serving radix cache. |
| Graph policy | The adapter must stop `one_batch.main` from raising the decode graph batch limit. | The customer's configuration, untouched. |

Evidence: load_model, dummy cache in extend, one_batch.main, run_scheduler_process.

FRAME 4 OF 12

## Limits are resolved and the grid agreed

Same rule

After `grid=[P, D] grid_digest=G`

BOTH DESIGNS

The grid is built from observed capacity and its digest is agreed before any point runs. An invalid explicit point fails. An invalid generated point is skipped and recorded.

| | A · Original | B · In-scheduler |
|---|---|---|
| Limits | Native pool and graph limits (design doc, page 4). | Adds the scheduler's own limits: `chunked_prefill_size`, `max_prefill_tokens`, `max_running_requests`, `prefill_max_requests`. |
| Point schema | A new `CaseSpec` record. | Dynamo's `benchmark_points.py` schema and digest helpers, moved to shared code. |

Evidence: prefill batch limits.

FRAME 5 OF 12

## Global warmup

Same rule

BOTH DESIGNS

Warmup timings are discarded. The executed count is recorded, not only the requested one.

| | A · Original | B · In-scheduler |
|---|---|---|
| Mechanism | The controller runs warm shots through the adapter. | One real request through the scheduler, as vLLM does: a 256-token prefill, then the configured number of decode steps. |

FRAME 6 OF 12

## P seeds the prefix, untimed

Same state

Before `cache={} live=[]`

After `cache={S1:512, S2:512} live=[]`

BOTH DESIGNS

A real, untimed prefill puts 512 tokens per request in the cache.

| | A · Original | B · In-scheduler |
|---|---|---|
| How | Build `S1`, `S2`, call `prepare_for_extend` and `ModelRunner.forward`, then checkpoint into the separately built cache. | Submit `S1`, `S2` through the front door with one output token. The stock result path checkpoints them into the radix cache. |
| Provenance label | "Real prefix", kept distinct from synthetic state. | `real_prefix`, one of the four values AISim already validates. |

Evidence: release_kv_cache, build_kv_cache.

FRAME 7 OF 12

## P warm shot, measured shot, release

Changed

```
Before  cache={S1:512, S2:512} kept=[] done=0/2
After   cache={S1:512, S2:512} live=[] kept=[P] done=1/2
```




Drawn to scale. 2 × 640 = 1,280 tokens = 1,024 matched + 256 computed in the measured forward.

BOTH DESIGNS

A warm shot, then the measured shot. Each request must match exactly 512 cached tokens. The 128-token suffix is never inserted into the cache. Both designs flush afterwards, so `cache={}` before `D`.

| | A · Original | B · In-scheduler |
|---|---|---|
| Execute | Fresh `M1`, `M2`, match 512 each, `prepare_for_extend`, forward. | Send a block message, submit `M1`, `M2` through the front door, wait until both are marked dispatched, then send unblock. SGLang's input blocker holds the two requests and hands them to the scheduler in one receive step, on every rank at once. The next loop step forms one prefill batch. Stock `run_batch` and `process_batch_result` do the rest. |
| Timed interval | Synchronized wall time around the call, or the DeviceTimer span. | Scheduler wall time, from `batch.launch_ts` to the moment the result reaches the host. The recorder waits on the same `copy_done` event the stock code waits on. Native GPU time is kept as raw evidence only. |

| | A · Original | B · In-scheduler |
|---|---|---|
| Actual shape | Checked by the new prefix provider. | The batch log shows one extend batch holding exactly `M1`, `M2`. Each response reports 640 prompt tokens, 512 of them cached. Native forward-pass metrics are a cross-check. |
| Keeping the suffix out | Release without insert. | Each shot gets a different first suffix token, so a later shot can match only the 512-token prefix. |

Evidence: run_batch, process_batch_result, shape metrics, input blocker, its barrier across ranks, where it runs, copy_done.

FRAME 8 OF 12

## D seed and admission step, not retained

Same arithmetic

```
Before    cache={} live=[] kept=[P] done=1/2
After     live=[D1:512, D2:512] kept=[P] done=1/2
```

BOTH DESIGNS

The admission step reads 511 past tokens per request and ends at sequence length 512, which fills page 32. Its timing is dropped.

| | A · Original | B · In-scheduler |
|---|---|---|
| How | Untimed `extend` on the 511-token prompts, then one `decode`. | Block, submit `D1`, `D2` with three output tokens and EOS ignored, unblock. Stock prefill runs, then the first decode step. |

FRAME 9 OF 12

## D steady step, release

Changed

```
Before    live=[D1:512, D2:512] kept=[P] done=1/2
After     live=[] kept=[P, D] done=2/2
```




from the 511-token prompt · token 512, written by the admission step · token 513, written by the steady step

One request's KV slots at the page boundary. The steady step is the first forward to touch page 33.

BOTH DESIGNS

Each request reads 512 past tokens, moves to length 513 and opens page 33. This is the retained sample: 2 requests, 1,024 past KV tokens.

| | A · Original | B · In-scheduler |
|---|---|---|
| Execute | A second direct `decode`. | The next scheduler loop iteration. |

| | A · Original | B · In-scheduler |
|---|---|---|
| Timed interval | Synchronized wall time around the call. | With overlap scheduling on, the time between the admission step's output and the steady step's output, the counterpart of vLLM's `inter_output`. With it off, launch to output. |
| Coordinate | Computed by the runner: 512 + 512 = 1,024. | Set by position: the group's second decode batch reads 511 + 1 past tokens per request, so 1,024. Native metrics report 1,026, because `prepare_for_decode` has already advanced the length. That value is only a cross-check, against an offset measured during warmup. |

In lane B the admission step natively reports 512 + 512 = 1,024. Selecting the sample by the native value would keep the wrong step, so the recorder selects it by position in the batch log.

Evidence: prepare_for_decode advances the length at line 3787. The offset is measured at warmup on every run, so a change in SGLang fails a check instead of shifting a coordinate.

FRAME 10 OF 12

## Reduce and write rank evidence

Changed

BOTH DESIGNS

Raw evidence is kept. One retained sample per point.

| | A · Original | B · In-scheduler |
|---|---|---|
| Records | New `RawSample` and `CaseResult` records and a new recorder. | Dynamo's rank-artifact schema v2, unchanged: `results`, `iteration_groups`, `run_id`, `grid_digest`, `coverage`, `timing`. Each point carries one record with the planned coordinates and the scheduler wall time. Native metrics sit under raw evidence. |
| Rank reduction | To be specified for each timing policy. | Group time is the maximum over ranks, as today. |

FRAME 11 OF 12

## The worker finalizes

Same in practice

BOTH DESIGNS

AISim treats the worker as disposable and stops it.

| | A · Original | B · In-scheduler |
|---|---|---|
| End state | Write artifacts, tear down, exit. | Flush the cache, write the artifact, then stop. |
| Serving handoff | A fresh worker only. | Possible in the same worker, still optional. |

FRAME 12 OF 12

### AISim validates and publishes Changed

After
```
row P: batch_size=2 total_prefill_tokens=256 total_kv_read_tokens=1024
row D: batch_size=2 total_prefill_tokens=0 total_kv_read_tokens=1024
```

BOTH DESIGNS

Complete coverage and matching identity are required before publication.

| | A · Original | B · In-scheduler |
|---|---|---|
| Validation | "AISim plus backend extension". | The existing `native_artifact.py` checks, with the `runtime.backend == "vllm"` test at line 383 switched on backend. |
| Per-point check | New. | `_expected_scheduled` at line 416 already requires exactly the `P` and `D` shapes above. |
| Timing policy | A new standalone policy in the writer and the Rust reader. | The existing `dynamo_native_single_sample_v1` policy, which the Rust reader accepts for any backend. No reader change. The timing rule is written into the artifact's `measurement_protocol`. |

## Unhappy paths

Each branch starts from a frame above. In the cache-miss and deadline branches both designs end the same way: the cell is incomplete, AISim keeps the evidence and does not publish it as complete.

BRANCH FROM FRAME 4

### An explicit point asks for 16,384 new tokens while the prefill chunk is 8,192

| | A · Original | B · In-scheduler |
|---|---|---|
| Outcome | The point passes pool and graph limits and runs as one forward, a shape chunked serving would not produce. | Rejected at grid build. If it slipped through, the scheduler would chunk it and the shape check would fail. |

BRANCH FROM FRAME 7

### `M2` matches 496 cached tokens, not 512

| | A · Original | B · In-scheduler |
|---|---|---|
| Detection | The prefix provider marks the point invalid. | The response for `M2` reports 496 cached tokens, and native metrics show 1,008 prefill KV tokens instead of 1,024. The driver re-stages once, as vLLM does. |

BRANCH FROM FRAME 7, WITH DP ATTENTION

### Two DP ranks do not launch the point in the same step

| | A · Original | B · In-scheduler |
|---|---|---|
| Outcome | Needs a new barrier across rank processes. | Block and unblock go to every DP rank, and the release waits at a barrier over all ranks, so every rank receives its requests in the same step. This holds with local control broadcast on or off. The batch log's per-rank token counts still check each point, and the driver re-stages on a mismatch. |

BRANCH BEFORE FRAME 8

### The soft deadline fires

BOTH DESIGNS

Stop at the point boundary. The attempt is partial, with `P` done and `D` missing. Resume stays bound to the frozen plan.

BRANCH FROM FRAME 7, WITH TP ABOVE 1

### One rank's forward raises

| | A · Original | B · In-scheduler |
|---|---|---|
| Outcome | Needs new failure propagation. `one_batch.main` only joins its rank processes. | The stock path signals the parent and the worker dies. AISim marks the attempt failed. |

Evidence: rank processes in one_batch.main, scheduler failure path.

## Invariant ledger

| RULE | ESTABLISHED | CHECKED | A AGAINST B |
|---|---|---|---|
| Actual shape equals planned shape | Frame 4 | Frames 7, 9, 12 | Same rule. B proves it from the batch log and each response. |
| Setup, admission and warmup never enter a retained sample | Frames 5, 6, 8 | Frames 7, 9 | Same |
| State provenance is truthful | Frames 6, 8 | Frame 12 | Same. B uses AISim's existing labels. |
| Grid and point order are agreed | Frame 4 | Frame 12 | Same |
| Everything is released before the next point | Frames 7, 9 | Frame 11 | Same rule. B uses the stock release path. |
| Complete coverage before publication | Frame 1 | Frame 12 | Same |

## Version exposure

The SGLang functions each design depends on were compared across three revisions: main at 5 October 2026, v0.5.22 and v0.5.23. Between the two tags the design targets, neither design breaks. They differ in how much they are exposed.

| DEPENDS ON | CHANGE ACROSS THE THREE REVISIONS |
|---|---|
| Lane A: `one_batch.py` helpers (`load_model`, `extend`, `decode`) | None. The file is identical in all three. |
| Lane A: the nine internal functions it must call directly | Five changed signature. Four only gained optional parameters. `release_kv_cache` broke callers: `is_insert` became a required `checkpoint` keyword. |
| Lane B: nine scheduler and receiver methods | Signatures identical in all three. |
| Lane B: the seams it installs and drives through (plugin loader, `async_generate`, the input blocker, `launch_ts`, `copy_done`) | Present in both tags. The plugin loader, the hook registry, the input blocker and its barrier are identical files in both. |
| Lane B: native forward-pass metrics | Fields identical in all three. `wall_time` changed meaning in v0.5.23, so lane B does not publish it. |

Lane A is more exposed because `one_batch.py` never builds a real prefix cache and never releases KV, so the adapter has to call those internals itself. Lane B inherits method bodies: `scheduler.py` changed by 434 lines between the two tags, and a recorder that wraps `run_batch` and `process_batch_result` without replacing them gets those changes for free.

Lane A is the more flexible design in one respect. It can build any batch shape and any synthetic state directly, without Dynamo or a server. Lane B measures only shapes the

scheduler can form.

## Gaps in the original design that lane B closes

| GAP IN LANE A | WITH LANE B |
|---|---|
| Scheduler wall timing needs a scheduler execution driver (the design doc's own GAP) | Closed. The real scheduler loop runs the batch. |
| `one_batch.extend` uses a dummy cache, so the adapter builds a separate one | Closed. Lane B uses the serving radix cache. |
| `one_batch.main` raises the decode graph batch limit | Closed. The customer's configuration is untouched. |
| Shapes that chunked serving would not produce can still run | Closed. The scheduler enforces its own limits. |
| A rank failure needs new propagation code | Closed for TP. The stock failure path stops the worker. |
| A new launcher and new flags instead of the vLLM ones | Closed. The serving worker takes the same `--benchmark-*` flags. |

## Settling what was still open

Four gaps were untouched by lane B and three were new in it. Each now has a decision. Two were revised after a second source reading: the DP attention barrier and batch formation now use SGLang's input blocker, and the AISim item became a launch interface. The last column gives the evidence and names the acceptance check that covers what source reading cannot prove.

| OPEN ITEM | DECISION | EVIDENCE AND WHAT IS LEFT |
|---|---|---|
| UNTOUCHED BY B<br>AISim admits only vLLM | A launch interface in the collector, with a vLLM implementation and an SGLang one. Each owns its entry gates, worker command, GPU identity, engine flags, preflight, provenance and resolved-config evidence. Rank-artifact validation, the database rows and the reader stay shared. Formal collection only. | The coupling is wider than the six literal gates (`cli.py:27`, `planner.py:731`, `fpm_builder.py:72`, `:92`, `:226`, `native_artifact.py:383`). The collector also assembles vLLM flags in `runner.py` and imports the vLLM scheduler in its preflight, so a backend table would not be enough. See AISim launch layer. Check 6. |
| UNTOUCHED BY B<br>The Rust reader accepts a closed set of policies | No reader change. Publish under the existing `dynamo_native_single_sample_v1` policy, as the vLLM path does. The timing rule goes into the artifact's `measurement_protocol`, where vLLM already writes `scheduler_wall_time`. | The reader accepts that policy for any backend (`fpm_forward.rs:574`). This replaces the earlier review suggestion of a `timing_boundary` column, which would have needed a new policy variant in the reader. Check 6. |

| OPEN ITEM | DECISION | EVIDENCE AND WHAT IS LEFT |
|---|---|---|
| UNTOUCHED BY B<br>DP attention needs a barrier across ranks | SGLang's input blocker. A block message and an unblock message go to every DP rank, and the release waits at a barrier over all ranks. No separate synchronizer. The driver owns the grid, so the ranks have nothing to negotiate. | The DP controller sends block messages to every worker, with local control broadcast on or off, and the blocker releases only when every rank has arrived. Both files are the same in the two tags. Collective RPC was rejected: all DP leaders connect to one RPC socket, so a call reaches only one of them. Per point, the per-rank token counts from the stock DP sync prove alignment. Check 2. |
| UNTOUCHED BY B<br>The GPU timer differs between the two tags | The published time never uses it. Scheduler wall time is built from `batch.launch_ts` and the `copy_done` event. | Both exist in v0.5.22 and v0.5.23. Native GPU time is stored raw and labelled by kind: a sum of intervals in v0.5.22, a span in v0.5.23. Check 4. |
| NEW IN B<br>How requests get into the scheduler | Through the engine's front door. The driver calls `Engine.async_generate` with token ids, a request id and a DP rank. | Those parameters exist in both tags, and SGLang's scripted test harness starts requests the same way. Building scheduler-internal request objects was rejected: their fields change between releases, and outputs for requests the tokenizer manager does not know end in its error path. |
| NEW IN B<br>Forming one exact batch | Block, submit, confirm dispatch, unblock. The driver sends the two messages through the tokenizer manager, as SGLang's own batch path does. | On each rank's socket the order is block, that rank's requests, unblock, so no request arrives after the release. The driver unblocks only when the tokenizer manager has marked each request dispatched. The blocker adds one all-rank sync to every scheduler step. In the overlap loop that sync runs while the previous forward is on the GPU, so the published times should not move; check 7 measures it. An in-place pause was rejected: it is not acknowledged, and under DP attention it is not ordered against the requests. Proof per point is the batch log, the record SGLang's scripted harness keeps. Check 1. |
| NEW IN B<br>Decode count of 1,026 against 1,024 | Not used to select or label the sample. Position in the batch log selects it, and the planned value labels it. | The native value is a cross-check against an offset measured at warmup on every run. Check 3. |

## AISim launch layer

AISim's vLLM route has two halves. The launch half decides how the worker is started: gates, engine flags, GPU count, the worker command and the in-pod checks. The artifact half validates and stores what the worker wrote. The launch half is tied to vLLM well beyond the six literal gates, so it becomes an interface with one implementation per backend. The artifact half stays shared.

| AN IMPLEMENTATION OWNS | VLLM TODAY | SGLANG IMPLEMENTATION |
|---|---|---|
| Entry gates | Literal `vllm` checks in the collector CLI, the support CLI and schema, the planner and the FPM builder. | One lookup of the backend implementation. |
| Worker command | Lifted from the Kubernetes builder. It must be `python3 -m dynamo.vllm` with a token list (`fpm_builder.py:92`). | Built as a token list directly, as the Slurm builder already does (`slurm_builder.py:154`). The Kubernetes builder's SGLang worker is a shell script with no hook for extra flags. |
| GPU identity | tp × pp × dp × pcp, read from vLLM flag names (`fpm_builder.py:355`). | tp × pp under DP attention, because DP ranks subdivide the TP group. AISim's `sglang.rule` already counts it this way. |
| Engine flags | `--max-model-len` with vLLM's `-1` auto-fit, token and sequence limits, prefix caching, async scheduling and CUDA-graph capture (`runner.py:1230`). | `--context-length` with a real value, and `--attn-dp-size` in place of the deprecated `--enable-dp-attention`. The other equivalents are confirmed one by one. |
| In-pod preflight | Imports `dynamo.vllm.instrumented_scheduler` and checks its methods (`runtime/preflight.py:53`). | Checks the SGLang version, the recorder plugin and benchmark mode. |
| Provenance | Writes backend `vllm` and the `vllm` package version (`runner.py:506`). | Writes `sglang` and its package version. |
| Resolved-config evidence | Files from vLLM's `--dump-config-to`, read by `database.py` and `execution_evidence.py`. | The driver writes the engine's resolved arguments to the same place. |

The benchmark flags never appear in the Kubernetes manifest. The pod only stays alive, and the engine command lives in a generated `run.sh` that the collector starts inside it. That is why the worker command, not the manifest, is the seam.

Shared: rank-artifact validation, which already expects one artifact per DP rank, the database rows and the Rust reader policy. Three readers in that half still assume vLLM and need the same lookup: the provenance check (`native_artifact.py:381`), `execution_evidence.py:108` and the config markers in `database.py:76`.

The generator is already organised per backend for ordinary deployments, so SGLang is an extension there. The collector is not: about 350 lines of `runner.py` assemble vLLM flags, so that side is a refactor of a working path. Its guard is three assertion suites and a pinned plan hash. The option name `vllm_max_model_len` is inside that hash and keeps its name. Check 6 adds a byte comparison of the rendered vLLM files.

## Code map

**Lane A reuses**

- one_batch.py helpers
- prepare_for_extend and prepare_for_decode
- DeviceTimer

**Lane B reuses in SGLang**

- The plugin loader and class replacement
- async_generate, the public front door
- The event loops and flush_cache
- The input blocker and its barrier across ranks
- Native forward-pass metrics, as a cross-check

**Lane B reuses in Dynamo**

- `vllm/benchmark_points.py`, importable as it is
- The `--benchmark-*` flags, to register in `sglang/backend_args.py`
- The schema v2 writer (`instrumented_scheduler.py:7747`) and the rank merge (`worker_factory.py:497`), to move to shared code

**AISim, both lanes**

- `collector/fpm_forward/native_artifact.py`
- `collector/fpm_forward/database.py` and `planner.py`
- `perf_database/fpm_forward.rs`, the Rust reader
- `collector/fpm_forward/runner.py` and `generator/builders/fpm_builder.py`, behind the launch interface

## Acceptance checks

These replace the open questions. Each is a small GPU run, repeated on v0.5.22 and v0.5.23. Checks 2 and 7 need at least two GPUs.

1. **Staging.** With two requests, and again with the batch at the scheduler's request limit, the batch log shows exactly one extend batch holding the group's request ids.
2. **DP attention.** With two DP ranks, both launch the point in the same step, and the per-rank token counts in the batch log equal the plan. Repeated with local control broadcast on.
3. **Decode.** The batch log shows extend, decode, decode for the group. The native decode count minus the planned one equals the offset measured at warmup.
4. **Timing.** For one prefill point and one decode point, the published scheduler wall time is no shorter than the GPU time in a trace of the same step. The size of the gap is recorded, not assumed.
5. **Failure.** Killing one rank during a point stops the worker, and AISim marks the attempt failed.
6. **AISim.** The existing validator passes the produced rank artifacts, and the Rust reader loads the published table with backend `sglang`. For vLLM, the plan, `run.sh` and manifest are byte-identical before and after the launch refactor.
7. **Blocker cost.** For a fixed running batch, the steady decode step takes the same time with `SGLANG_ENABLE_COLOCATED_BATCH_GEN` set and unset, within run-to-run noise. Repeated under DP attention.

## Boundaries

- The specimen is TP 1 without DP. The DP path is designed (the DP branch above and check 2), but the walkthrough does not trace it.
- Version one measures real states only: a real prefix and real decode KV. AISim already asks vLLM for real seeding (`config.py:80`), so the synthetic `fake_prefix` and `fake_fallback` regimes are left out.
- DP attention is supported with local control broadcast on or off. DWDP and elastic EP are rejected at start, because the vLLM self-benchmark covers neither. Attention context parallelism above 1 is rejected too, because the input blocker's rank check does not cover it.
- Under DP attention SGLang divides the chunked prefill size by the DP size. The planner checks each prefill point against that per-rank budget, or the point would split across two steps.
- The input blocker is a little-used SGLang feature: one caller in the batch path, one line in the docs, and no test in the tree that was read. Checks 1, 2 and 7 are its evidence here.
- Multi-node SGLang collection is not designed. The vLLM path supports it through vLLM's own orchestration flags.
- AISim's runtime probe, instrumentation and repeatability paths stay vLLM-only. Formal collection does not need them.
- The plugin loader logs a failed hook instead of raising. The driver therefore fails the run if any rank's recorder evidence is missing.

---

Sources read: SGLang at the v0.5.22 and v0.5.23 tags and at main from 5 October 2026, Dynamo at 95b9d56 and AISimulate at 925c66ec. SGLang line numbers are for v0.5.23. The Dynamo and AISim revisions are not the ones the design doc pins, so their line numbers may differ slightly. Specimen values are illustrative. The AISim launch-layer facts come from a survey of that checkout, with the cited lines read again. Nothing in this document was run on a GPU.